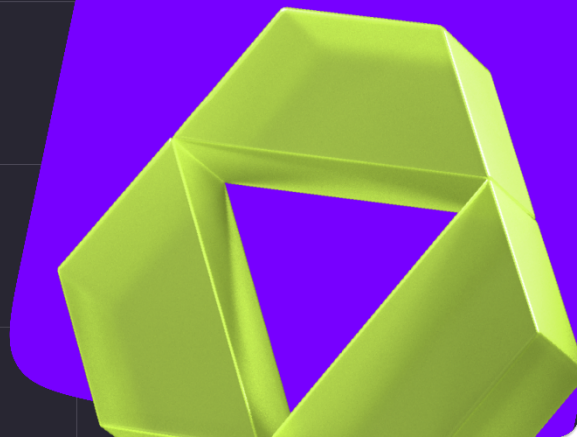


Постридер

SQL для начинающих

Занятие 10



Теперь вы знаете, как создавать таблицы: задавать типы данных, ограничения и первичные ключи.

DDL-команды: CREATE, ALTER, DROP, TRUNCATE

Разобрали четыре ключевые DDL-команды и чем они отличаются от DML.

CREATE — создаёт новый объект (таблицу, схему, индекс).

ALTER — изменяет структуру существующего объекта (добавить/удалить столбец, изменить тип).

DROP — полностью удаляет объект вместе с данными. Необратимо!

TRUNCATE — быстро удаляет все строки из таблицы, но структура остаётся. Работает быстрее **DELETE**.

CREATE TABLE: типы данных и ограничения

Изучили синтаксис создания таблицы с ограничениями на уровне столбца и на уровне таблицы.

```
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,      -- автоинкремент + уникальность  
  name VARCHAR(100) NOT NULL, -- обязательное поле  
  age INT NOT NULL            -- только целые числа  
    CHECK (age > 0 AND age < 100)  
    DEFAULT 18,                -- значение по умолчанию  
  field VARCHAR(100),  
  UNIQUE(name, age)            -- составная уникальность  
);
```

SERIAL — это не просто тип данных, а так называемый автоинкрементный счётчик. При каждой вставке значение автоматически увеличивается на 1.

Ограничения на уровне таблицы (**UNIQUE**(col1, col2), **CHECK** с несколькими полями) пишутся после всех столбцов через запятую.

● Пример: задача 1 — таблица clients

Создать таблицу клиентов Ростелекома с уникальным числовым ID, обязательным именем, необязательной фамилией и типом авторизации.

```
CREATE TABLE clients (  
  user_id  INTEGER UNIQUE NOT NULL,  
  firstname TEXT  NOT NULL,  
  lastname TEXT,  
  auth_type TEXT  
);
```

Обратите внимание, здесь использована комбинация **UNIQUE + NOT NULL** вместо **PRIMARY KEY**.

● Пример: задача 2 — таблица projects

Создать таблицу проектов с автоинкрементным ключом, датами и составным ограничением уникальности.

```
CREATE TABLE projects (  
  project_id SERIAL PRIMARY KEY,  
  manager   VARCHAR(50),  
  project_name TEXT NOT NULL,  
  start_date DATE NOT NULL,  
  end_date  DATE CHECK (end_date >= start_date),  
  UNIQUE (manager, project_name)  
);
```

CHECK (end_date >= start_date) — ограничение между столбцами. Оно сравнивает два столбца между собой, поэтому должно быть задано на уровне таблицы.

Если **end_date** не заполнен (**NULL**), проверка пропускается — **NULL >=** дата возвращает **NULL**, а не **FALSE**. Это корректное поведение для незакрытых проектов.

● Пример: задача 3 — таблица members

Создать таблицу клуба лояльности с проверкой даты через **CURRENT_DATE** и ограничением допустимых значений через **CHECK** с **IN**.

```
CREATE TABLE members (  
  member_id SERIAL PRIMARY KEY,  
  full_name TEXT NOT NULL,  
  join_date DATE CHECK (join_date <= CURRENT_DATE),  
  membership_type TEXT CHECK (membership_type IN ('standard', 'premium', 'lifetime')),  
  email TEXT UNIQUE  
);
```

CURRENT_DATE — это не константа, а функция PostgreSQL. При каждой проверке она возвращает текущую дату. Это значит, что сегодня допустимо значение "сегодня", а завтра — нет.

CHECK с IN — удобный способ ограничить набор допустимых значений. Аналог перечисления: значение должно быть одним из указанных.

Важные нюансы

- **SERIAL — это синтаксический сахар**

За удобным словом **SERIAL** скрывается более сложная конструкция: **PostgreSQL** автоматически создаёт последовательность (**SEQUENCE**) и привязывает её к столбцу.

- Запись **id SERIAL PRIMARY KEY** эквивалентна: создать **SEQUENCE**, а в столбце **id** задать **DEFAULT nextval(sequence)**. Это важно знать при импорте данных вручную.

- Если вставить **ID** вручную, минуя **SERIAL**, счётчик не сдвинется — и при следующей автоматической вставке возможен конфликт уникальности.

- **Составной PRIMARY KEY**

Первичный ключ может состоять из нескольких столбцов — тогда уникальной должна быть их комбинация.

```
CREATE TABLE students (  
    id SERIAL,  
    email VARCHAR(100),  
    name VARCHAR(100),  
    PRIMARY KEY(id, email)  
);
```

Составной ключ означает: допускается один и тот же **id** с разными **email**, или одинаковый **email** с разными **id**, но не одновременно одинаковые **id** И **email**.

- **ALTER TABLE — изменение структуры после создания таблицы**

Если таблица уже существует, её структуру можно изменить командой **ALTER TABLE** без потери данных.

```
ALTER TABLE students ADD COLUMN phone VARCHAR(20);  
ALTER TABLE students DROP COLUMN phone;  
ALTER TABLE students ALTER COLUMN age TYPE SMALLINT;  
ALTER TABLE students ADD CONSTRAINT unique_email UNIQUE (email);
```

- **Разница между DROP и TRUNCATE**

Обе команды удаляют данные, но работают по-разному — важно не перепутать.

DROP TABLE students — удаляет таблицу полностью вместе со структурой. Восстановить невозможно.

TRUNCATE TABLE students — удаляет все строки мгновенно, структура таблицы остаётся. Быстрее **DELETE**.

DELETE FROM students — удаляет строки по одной, можно с **WHERE**, активирует триггеры, можно откатить.